

Liqaa | لقاء

Smart Meeting Point Recommendation Platform

Java & Spring Boot Capstone Project · Tuwaiq Academy

Presented by: **Nawaf Alghamdi**



The Problem & Our Solution

Problem

When a group wants to meet, choosing a suitable location can become difficult.

- Participants live in different areas.
- A place may be close to one person but far from another.
- Comparing locations manually takes time.
- Larger groups make the decision more difficult.
- Choosing a location based only on the geographic center does not guarantee fairness.

"Where should everyone meet?"

Solution

Liqaa coordinates the complete meeting process.

It allows users to:

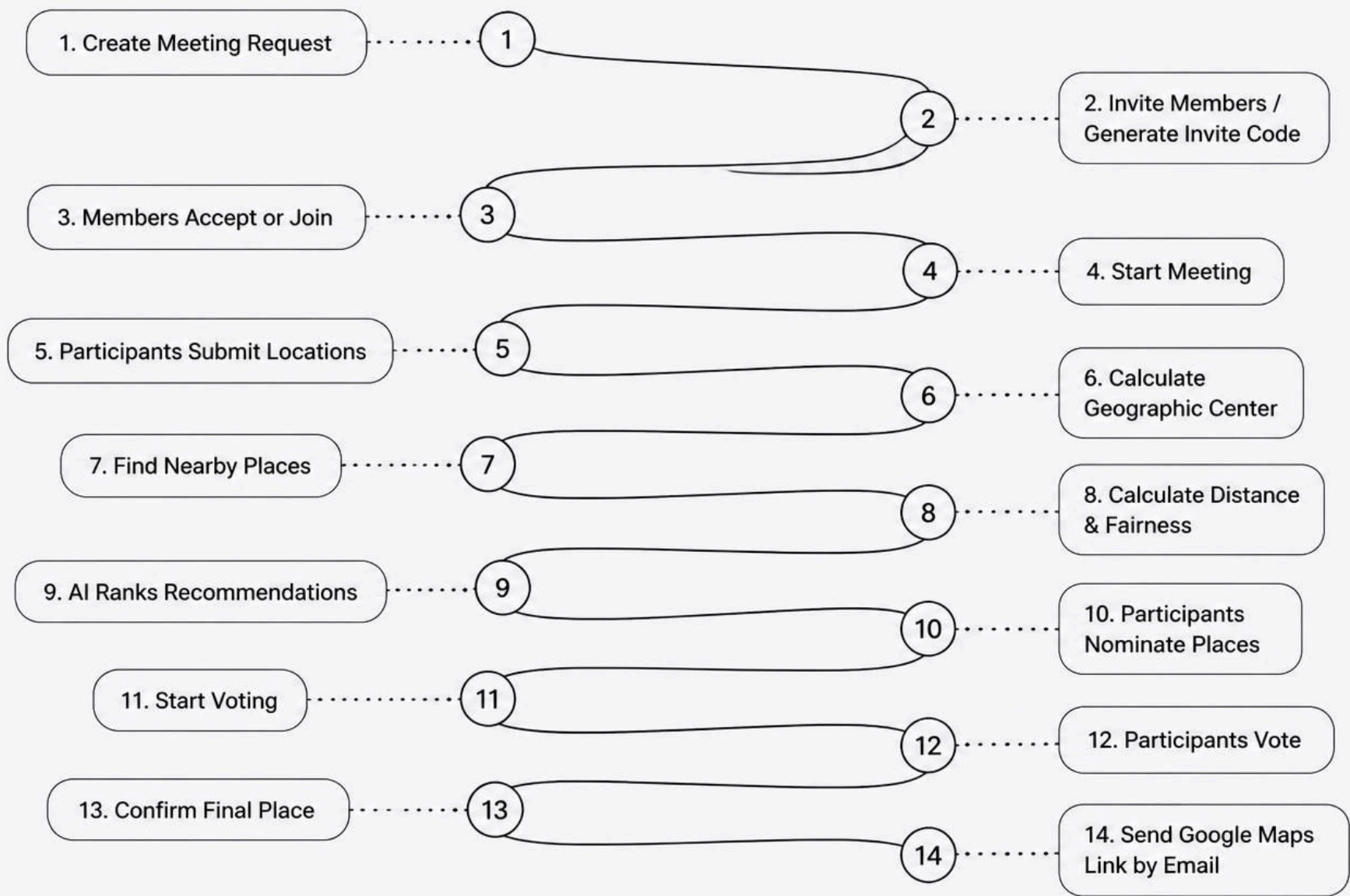
- Create a meeting request.
- Invite members.
- Share an invite code.
- Collect participant locations.
- Calculate the group center.
- Discover real nearby places.
- Measure distance and fairness.
- Rank places using AI.
- Vote on candidate places.
- Confirm the final meeting location.

Participants

Liqaa

Fair Meeting

How Liqaa Works



Meeting Request Lifecycle

PENDING → READY / CANCELLED

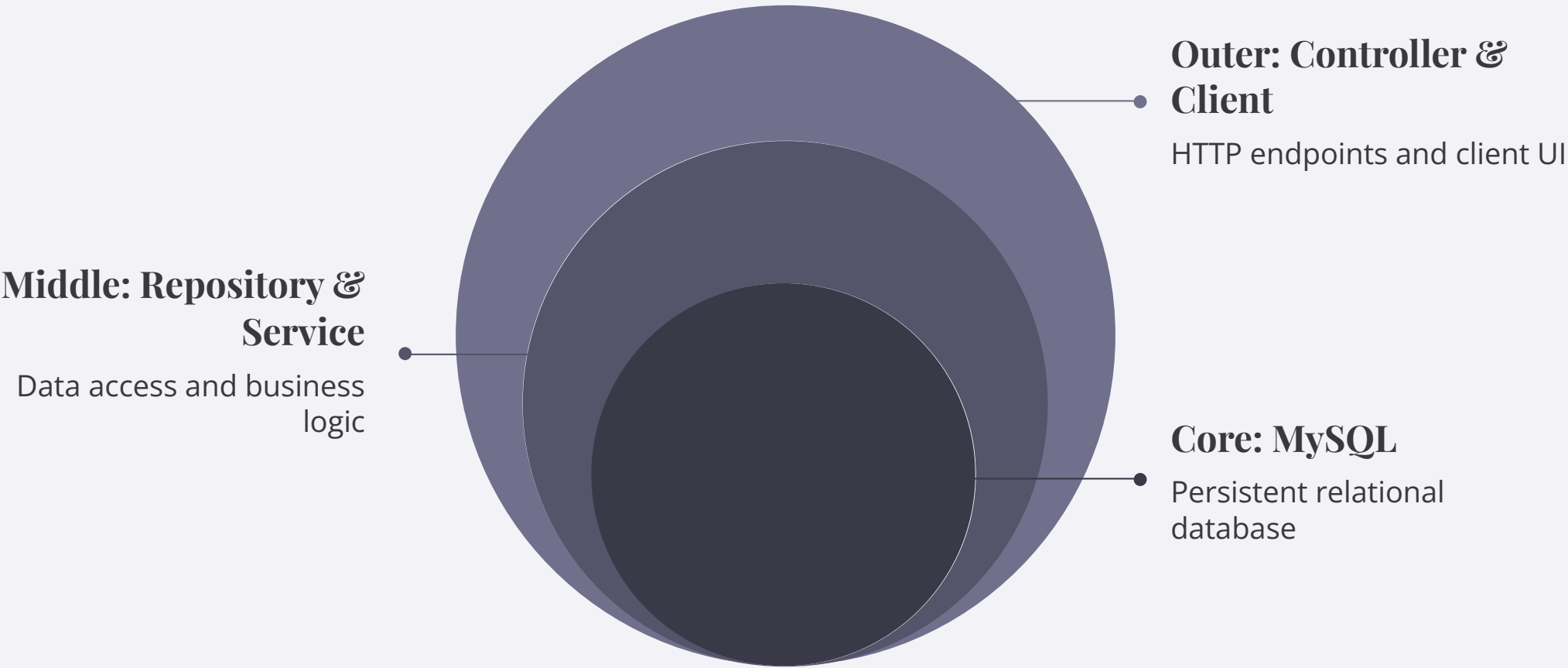
Meeting Lifecycle

OPEN → VOTING → CONFIRMED / CANCELLED

The organizer controls administrative meeting actions, while participants control their own location and votes.

System Architecture

Liqaa follows a layered Spring Boot architecture.



Layer Descriptions

- **Controller** — Handles REST requests and HTTP responses.
- **Service** — Contains business logic, authorization, calculations, and meeting workflow.
- **Repository** — Handles database operations using Spring Data JPA.
- **Model** — Contains JPA entities.
- **DTO** — Defines structured request, response, and external API data.
- **Client** — Communicates with external services.

External Integrations

MeetingService

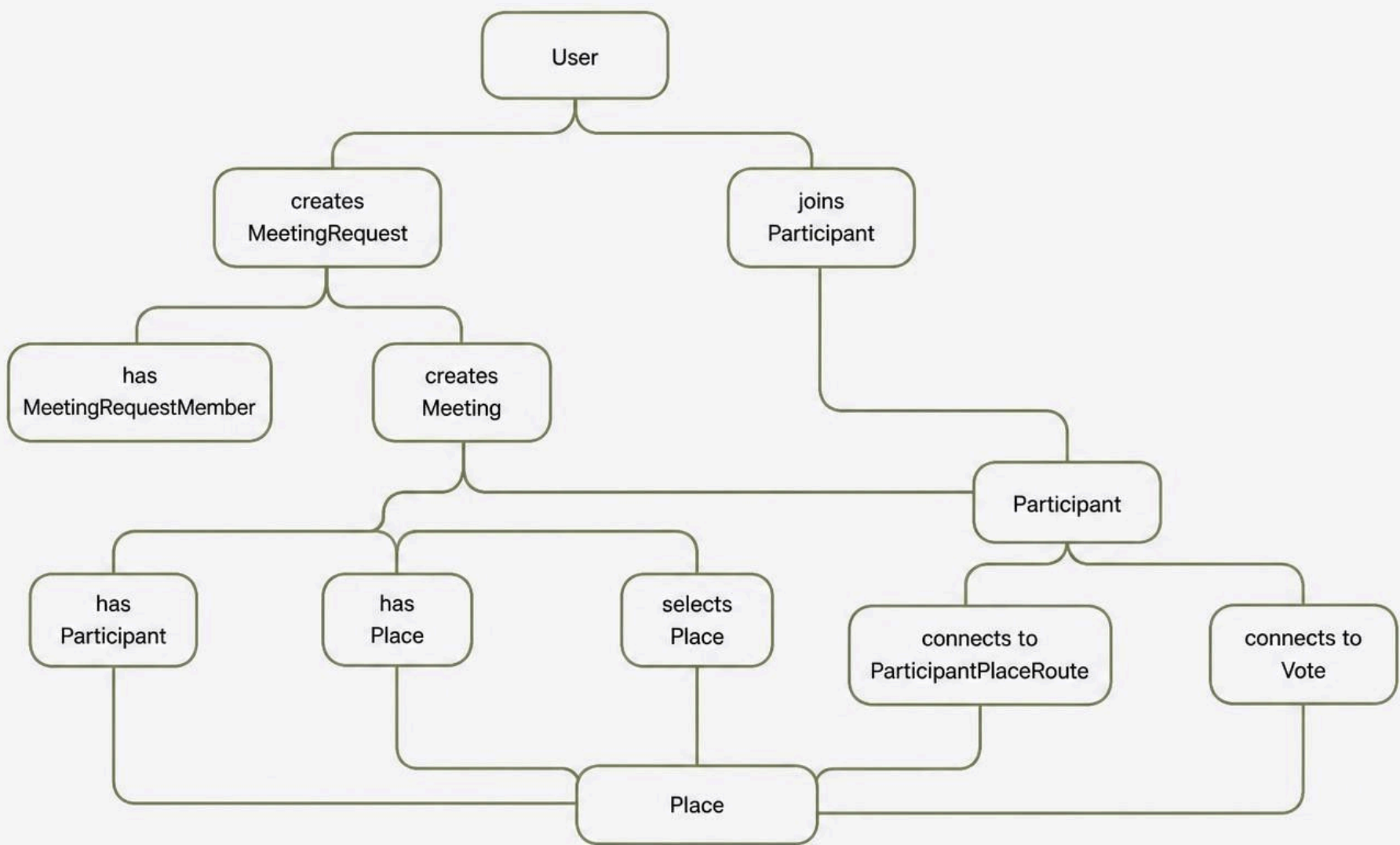
- GooglePlacesClient
- OpenRouterClient
- DistanceService
- FairnessService
- EmailService

❏ External APIs are separated from the main business logic through dedicated client classes.

Database Design

The 8 main tables:

- User
- MeetingRequest
- MeetingRequestMember
- Meeting
- Participant
- Place
- ParticipantPlaceRoute
- Vote



User	Stores registered users.
MeetingRequest	Stores the organizer, category, meeting time, invite code, and request status.
MeetingRequestMember	Stores invited users and invitation status.
Meeting	Stores the active meeting, center coordinates, status, and selected place.
Participant	Stores users participating in the meeting and their coordinates.
Place	Stores candidate places selected for voting.
ParticipantPlaceRoute	Stores calculated participant-to-place distances.
Vote	Stores each participant's selected place.

Finding a Fair Meeting Area

Step 1 — Geographic Center

Each participant submits:

- Latitude
- Longitude

The backend calculates:

Center Latitude = Sum of Participant Latitudes / Number of Participants

Center Longitude = Sum of Participant Longitudes / Number of Participants

Example:

- Participant A → 24.70, 46.60
- Participant B → 24.80, 46.70
- Participant C → 24.75, 46.65

Center → 24.75, 46.65

Step 3 — Metrics

For every place, Liqaa calculates:

Average Distance Average distance between all participants and the place.	Maximum Distance Distance of the participant who is farthest from the place.	Fairness Score Measures how evenly participant distances are distributed.
---	--	---

Fairness Formula

Fairness Score = $100 - (((\text{Max Distance} - \text{Min Distance}) / \text{Max Distance}) \times 100)$

Example

Place A: 5 km | 5 km | 5 km | 5 km → Fairness = 100

Place B: 1 km | 2 km | 3 km | 10 km → Fairness = 10

Step 2 — Distance

After Google Places returns candidate places, Liqaa calculates the distance from every participant to every place.

The system uses the **Haversine Formula**.

Haversine calculates straight-line geographic distance in kilometers, not driving distance or travel time.

Important

A high fairness score does not automatically mean the place is best.

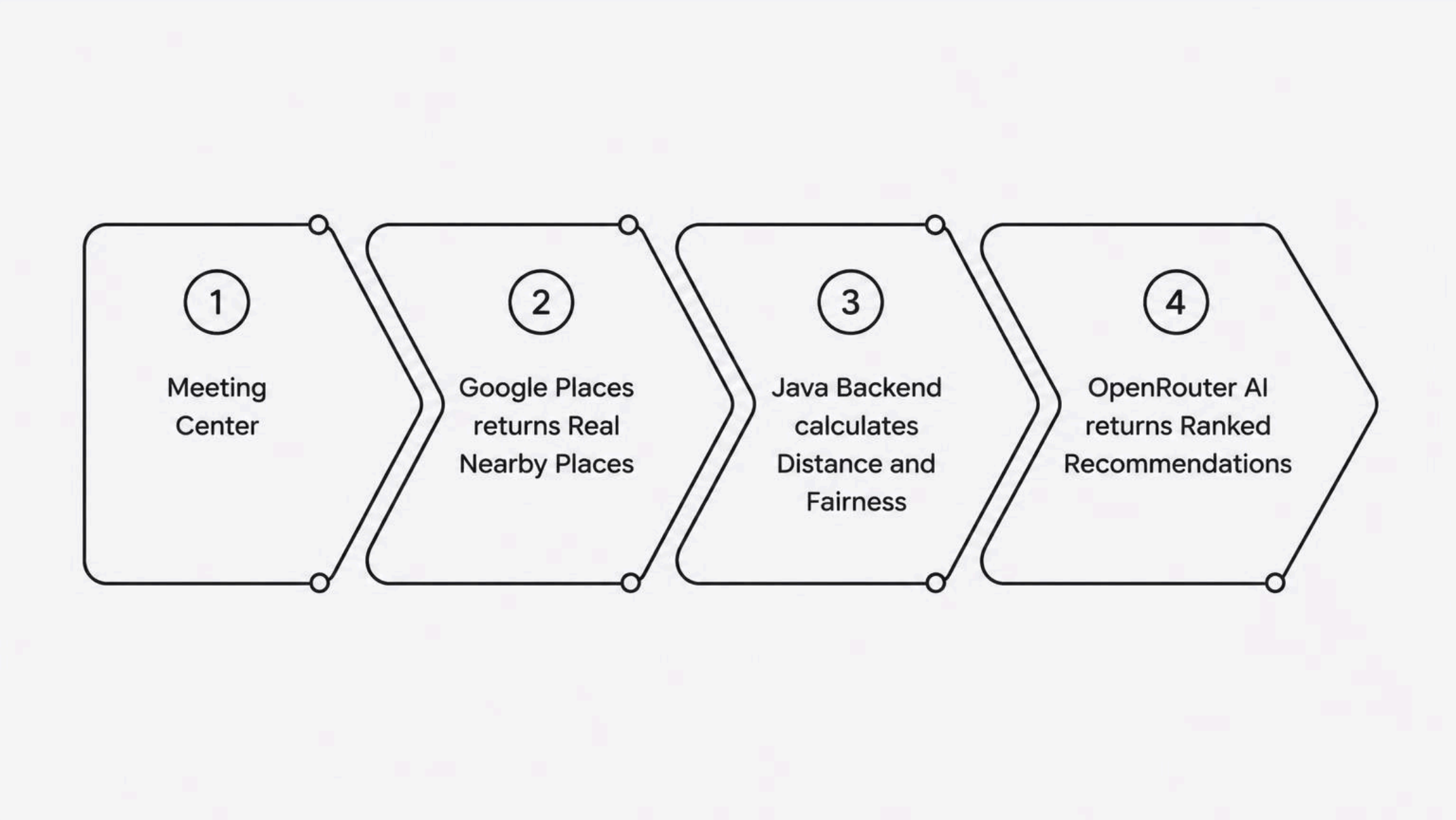
For example:

10, 10, 10, 10 km and 2, 2, 2, 2 km

both have perfect fairness, but the second place is much closer.

This is why Liqaa evaluates multiple metrics.

Smart Place Recommendations



Google Places

Google Places receives:

- Meeting center
- Meeting category

Example categories:

- Cafe
- Restaurant
- Park
- Movie Theater
- Shopping Mall

It returns real locations with:

- Google Place ID
- Name
- Address
- Coordinates
- Rating

Java Backend

For each place, Java calculates:

- Participant distances
- Average distance
- Maximum distance
- Fairness score

OpenRouter AI

AI receives the real places and their calculated metrics.

It returns:

- AI Rank
- AI Score
- Recommendation Reason

Google Places	Java Backend	OpenRouter
Finds real places	Calculates center, distance, and fairness	Analyzes and ranks the results

📌 AI does not invent places and does not perform the geographic calculations.

From Recommendations to Final Decision

Recommendations

Recommendations are initially temporary.

Participants can nominate recommended places into the voting pool.

When a place is nominated:

- The place is stored.
- Participant distances are recalculated by the server.
- Distance records are stored for that place.

Voting

The organizer starts voting.

Meeting status changes: OPEN → VOTING

Each participant can:

- Submit one vote.
- Change their own vote.

Example:

- Cafe A → 3 Votes
- Cafe B → 1 Vote
- Cafe C → 1 Vote

Cafe A wins.

Authorization

Organizer

- Manages meeting requests.
- Manages invitations.
- Starts/cancels meetings.
- Manages participants.
- Calculates the meeting center.
- Starts voting.
- Confirms the meeting.

Tie Break

If places have the same number of votes, the Fairness Score is used.

Example:

- Cafe A — Votes: 2, Fairness: 82
- Cafe B — Votes: 2, Fairness: 91

Cafe B is selected.

Confirmation

After all participants vote, the organizer confirms the meeting.

The system:

- Stores the selected place.
- Changes status to CONFIRMED.
- Generates a Google Maps link using the Google Place ID.
- Emails the final location to participants.

Participant

- Updates only their own location.
- Nominates places.
- Submits only their own vote.
- Changes only their own vote.

Core API Endpoints

Meeting Management

Endpoint	Description
Create User	Creates a registered Liqaa user.
Create Meeting Request	Creates a new meeting request for an organizer.
Generate Invite Code	Creates a shareable code for joining the meeting request.
Invite Member	Sends an invitation to a member by email.
Join by Invite Code	Allows a registered user to join using an invite code.
Start Meeting	Creates the meeting and converts accepted members into participants.

Location & Recommendation

Endpoint	Description
Update Participant Location	Stores the participant's current latitude and longitude.
Check Location Readiness	Checks whether all participants have submitted their locations.
Calculate Meeting Center	Calculates the geographic center of participant locations.
Generate Recommendations	Retrieves real places, calculates distance and fairness, and applies AI ranking.
Select Place for Voting	Adds a recommended place to the meeting voting pool.

Voting & Confirmation

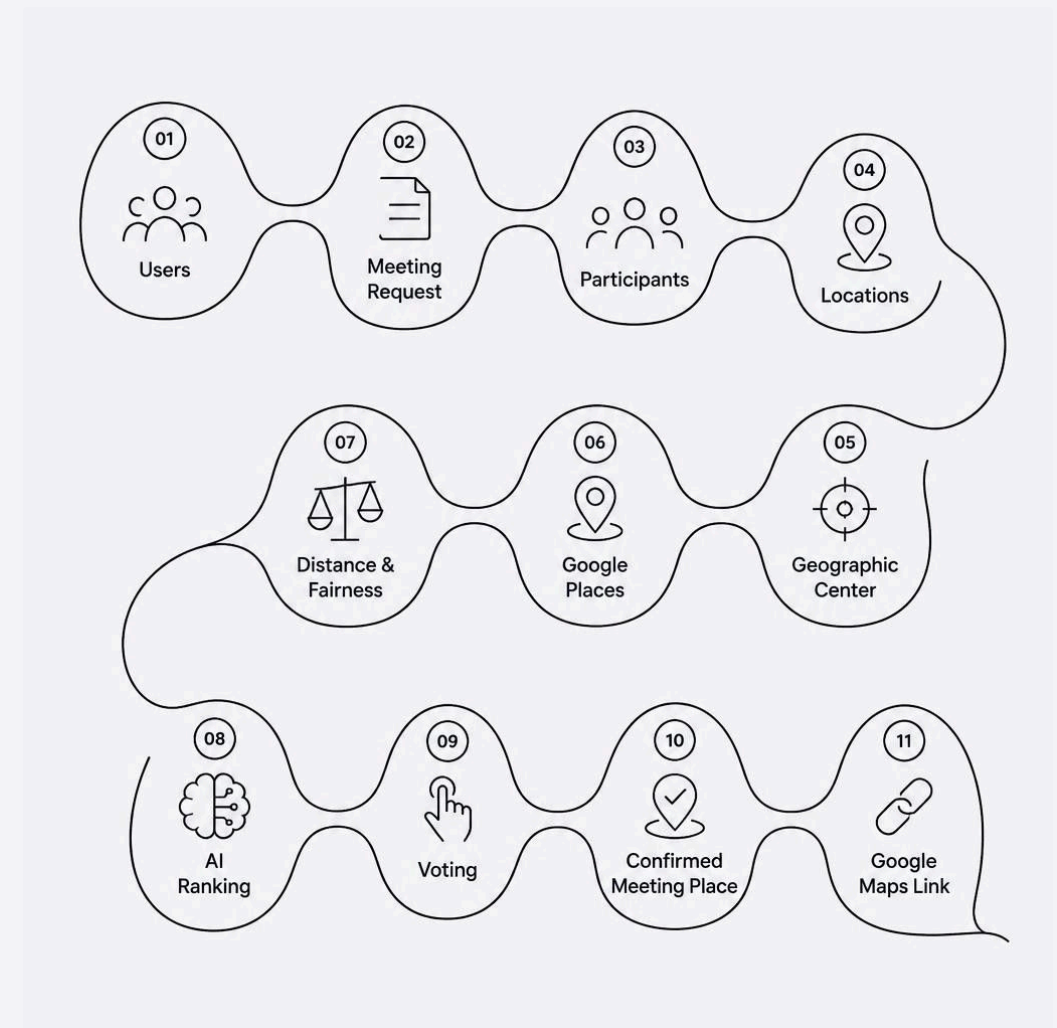
Endpoint	Description
Start Voting	Changes the meeting status from OPEN to VOTING.
Submit Vote	Allows a participant to vote for a candidate place.
Change Vote	Allows a participant to change their existing vote.
Get Voting Result	Returns vote counts and voting progress.
Confirm Meeting	Determines the winning place, confirms the meeting, and sends the final location.

Technology Stack & Project Overview

Final System Overview

Tech Stack

- **Language:** Java
- **Backend:** Spring Boot, Spring Web
- **Persistence:** Spring Data JPA, Hibernate
- **Database:** MySQL
- **Validation:** Jakarta Validation
- **AI:** Spring AI, OpenRouter
- **Location:** Google Places API
- **Communication:** Spring Mail
- **Tools:** Maven, Lombok



"Liqaa combines real location data, deterministic geographic calculations, fairness analysis, AI-assisted ranking, and participant voting to make group meeting decisions simpler and more balanced."